BEMS CONNECT · ENGINEERING HANDOVER

AI Group Financial Manager — backend contract

The conversational agent is built and deployed in n8n. It holds no database credentials and no authority of its own: every read and every write goes through BEMS backend endpoints, which authenticate the signed-in user and enforce all permissions and policy. This document is the list of endpoints the agent calls, the shape it expects back, the rules the backend must enforce, and the decisions still open.

AGENT RUNTIME	WEBHOOK PATH	ENDPOINTS REQUIRED
n8n · workflow mBIsnmPAtpCS38a4	POST /webhook/bems-financial-manager	23 (10 new, 4 to extend)

AUTHORISED ROLES

GMD · SYSTEM_ADMIN

CONTENTS

1. Request path
2. What BEMS sends the agent
3. What the agent returns
4. The staged-action model
5. Read endpoints
6. Staged-write endpoints
7. Error and permission envelope
8. Rules the backend must enforce
9. Settings and infrastructure
10. Open questions
11. Acceptance scenarios

01 Request path

The chat UI must never call the n8n webhook directly. BEMS backend is both the front door and the tool provider; n8n sits in the middle and carries no privilege.

```
GMD / System Admin in BEMS chat UI
↓
```

```
BEMS backend — authenticates user, checks role, mints/forwards token
↓
n8n webhook — orchestration and model calls only
↓
BEMS backend API — permissions, validation, policy, transactions, audit
↓
PostgreSQL
```

Every tool call the agent makes carries `Authorization: Bearer <user token>` and `X-Correlation-Id`. The backend authorises against that token on every single call — not once at the start of the conversation.

02 What BEMS sends the agent

One POST per chat message, to the n8n webhook. Protect it with a shared service secret at the network or gateway layer; it must not be reachable from the browser.

```
POST /webhook/bems-financial-manager
Content-Type: application/json

{
  "user": {
    "id":    "usr_8831",
    "name":  "Dr Pastra",
    "title": "Group Managing Director",
    "role":  "GMD"                 // GMD | SYSTEM_ADMIN
  },
  "session_id":     "conv_01HX7K2M…",   // server-derived, never client-supplied
  "message":        "What needs my attention today?",
  "auth_token":     "eyJhbGciOi…",       // forwarded to every tool call
  "correlation_id": "req_01HX7K2P…"      // also used as the idempotency key
}
```

HARD REQUIREMENT

`session_id` must be derived server-side from the authenticated user. If the client can supply it, one user can read another user's conversation history, which contains salary and banking data.

03 What the agent returns

Synchronous JSON. `200` on success, `403` if the caller is not GMD or System Admin, `503` if the model or a tool failed — in which case nothing was written and BEMS should tell the user to continue in the app.

```
{
  "ok": true,
  "reply": "Good morning, Dr Pastra. Three things need you today…",
  "as_of": "2026-10-20T09:14:22+01:00",
  "requires_confirmation": true,
  "staged_action": {
    "id": "sa_01HX7M…",
    "type": "budget",
    "summary": "Create Q1-2026 budget for BemsSoft Hub — ₦20,000,000 across 3 lines",
    "expires_at": "2026-10-20T09:29:22+01:00"
  },
  "citations": [
    { "record_type": "fund_request", "record_id": "REQ-2026-0010",
      "label": "Cloud renewal — ₦340,000", "url": "https://bems…/requests/REQ-2026-0010
  ],
  "session_id": "conv_01HX7K2M…",
  "correlation_id": "req_01HX7K2P…"
}
```

When `requires_confirmation` is true, render `staged_action` as a card with explicit **Confirm** and **Cancel** buttons that post the action id back as the next message. Do not rely on the user typing "yes" — a structured affordance removes a whole class of mis-confirmation risk and is the single highest-value thing the frontend can contribute here.

04 The staged-action model

This is the core safety mechanism and the piece most likely to be got wrong. No write is ever executed in one step. The agent stages, the user confirms, the backend executes.

Critically, **the confirmation state lives in the backend, not in the conversation.** The model must never be the thing that remembers a user said yes.

Staging response — returned by every `prepare_*` endpoint

```
{
  "staged_action_id": "sa_01HX7M…",
  "confirmation_hash": "sha256:4f2a…",   // hash of all material fields
  "type": "budget",
  "expires_at": "2026-10-20T09:29:22+01:00",
  "summary": "Create Q1-2026 budget for BemsSoft Hub — ₦20,000,000 across 3 lines",
  "preview": { /* full before-and-after the user is shown */ },
```

```
  "warnings": ["An existing Q1-2026 budget for this organization will be superseded"],
  "blocking_errors": []
}
```

Execution — `POST /ai/staged-actions/{id}/confirm`

Body carries the `confirmation_hash`. The backend recomputes the hash from the stored staged action and **rejects if it differs**, so a staged action can never execute against changed inputs. It also rejects if expired, already executed, or belonging to a different user.

```
{
  "ok": true,
  "record_type": "budget",
  "record_id": "BUD-2026-0031",
  "url": "https://bems…/budgets/BUD-2026-0031",
  "record": { /* the persisted record, read back after commit */ },
  "notifications_sent": ["usr_4412"],
  "audit_event_id": "aud_01HX7N…"
}
```

Return the committed record, read back after the transaction. The agent is instructed to report success only from this payload — it is forbidden from claiming a write landed on the strength of its own intent.

CONCERN	REQUIRED BEHAVIOUR
Material-field change	Hash mismatch → reject. Agent re-stages and re-asks.
Expiry	TTL on every staged action. Suggested: 15 minutes.
Replay	Honour `Idempotency-Key` (the correlation id). Second confirm returns the first result, does not re-execute.
Ownership	A staged action may only be confirmed by the user who staged it.
Single use	Mark executed atomically within the same transaction as the business write.

05 Read endpoints

EXISTS likely already present EXTEND exists but needs fields added BUILD new endpoint

GET `/ai/briefing` BUILD

`get_daily_briefing`

Pre-aggregated executive briefing. Deliberately one endpoint rather than letting the model stitch eight calls together — it is the most frequently hit path and the chattiest if left unaggregated.

query organization_id?
returns decisions awaiting this user · approved-but-unrecorded payments · payments due soon · awaiting recipient confirmation · budget risks and overspend · payroll actions and deadlines · overdue accountability · ranked next actions · as_of

---

GET **/fund-requests** EXISTS

search_fund_requests

query q · status · organization_id · requester_id · category · date_from · date_to · min_amount · limit
returns id · title · requester · subsidiary · category · amount · currency · status · due_date · age_days

---

GET **/fund-requests/{id}** EXTEND

get_fund_request_detail

Must return the full approval history and budget impact in one call — the agent is required to read this before staging any decision.

returns all list fields · vendor · budget line and remaining balance · full approval routing and history · attached evidence · disbursement status · recipient confirmation · accountability state · applicable policy checks

---

GET **/approvals/pending** EXISTS

get_pending_approvals

query assigned_to_me · organization_id?
returns request_id · amount · requester · subsidiary · waiting_days · due_date · urgency · approval_stage

---

GET **/budgets** EXTEND

get_budgets

Needs `include_utilisation` returning committed, spent, remaining and utilisation percentage per allocation line.

query budget_id? · organization_id? · period? · include_utilisation
returns budget · allocation lines · commitments · actual spend · remaining · utilisation pct

remaining · utilisation_pct

GET **/disbursements** EXISTS

search_disbursements

query disbursement_id? · request_id? · status? · organization_id? · date_from · date_to · limit
returns linked request · amount · payer account · beneficiary · bank · transfer_reference · payment_date · receipt file · recipient confirmation status · issues

GET **/accountability** EXISTS

get_accountability_status

query state? · organization_id? · overdue_only
returns case id · linked request and disbursement · evidence status · days overdue · outstanding follow-up

GET **/payroll/summary** EXTEND

get_payroll_summary

Aggregate figures only — never individual salaries, which would otherwise leak into briefings.

query period? · organization_id? · compare_previous
returns total cost · headcount · per-subsidiary and intercompany allocations · payment status · upcoming deadlines · issues

GET **/staff** EXISTS

search_staff

Name-to-id resolution. Must not return pay figures.

query q · organization_id?
returns employee_id · name · job title · organization

GET **/staff/{employee_id}/compensation** BUILD

get_staff_compensation

The most sensitive read in the system. Enforce permission strictly and log every access as an audit event, including reads.

returns current compensation · full versioned change history · approver and timestamp per change

GET **/vendors** EXISTS

`get_vendors_and_invoices`

query vendor_id? · q? · include_invoices · date_from · date_to
returns vendor details · invoices with amounts, due dates and payment status · recurring commitments · period spend total

GET **/analytics/financial** BUILD

`get_financial_analytics`

The analysis workhorse. The agent is instructed to use this rather than pulling raw lists and doing arithmetic itself — the backend owns every number the user is shown.

query dataset (requests · approvals · payments · payroll · budgets · vendors · accountability) · group_by (organization · department · category · requester · vendor · status · month) · date_from · date_to · compare_to_previous · organization_id? · flag_anomalies
returns grouped totals · prior-period comparison · variance · trend · anomaly flags (duplicate candidates, unusual amounts, repeated exceptions, late approvals)

GET **/analytics/cash-forecast** BUILD

`get_cash_forecast`

Must return the component obligations, not just a total, so the agent can show its working and label it a forecast.

query horizon_days · organization_id?
returns approved-unpaid requests · scheduled recurring commitments · payroll obligations · total by week · assumptions

GET **/organizations** EXISTS

`get_organizations`

query include_departments
returns organization_id · display name · departments · default currency

GET **/ai/verify** BUILD

`verify_action_result`

Re-reads a record to confirm a write persisted. Backs the "never claim unverified success" rule when a confirm call times out or returns ambiguously.

query record_type · record_id? · staged_action_id?
returns exists · the record · notification delivery status

POST **/reports/export** EXTEND

generate_report_export

Read-only despite the verb — exports existing authorised data and changes nothing, so it is not staged.

body report_type · format (pdf · xlsx · csv) · date_from · date_to · organization_id? · title
returns download url · expires_at

06 Staged-write endpoints

All seven are new. Each `prepare_*` validates and returns the staging response from section 4 without writing anything.

POST **/ai/staged-actions/budget** BUILD

prepare_budget

Validates that allocation lines sum to the total, checks for an existing budget for that organization and period, and warns rather than silently superseding.

body mode (create · update) · budget_id? · organization_id · period · currency · total_amount · allocations[{line_name, amount}] · note?

POST **/ai/staged-actions/approval-decision** BUILD

prepare_approval_decision

Must reject outright if the decision is not assigned to the signed-in user. Reason is mandatory for DECLINE and REQUEST_CLARIFICATION.

body request_id · decision (APPROVE · DECLINE · REQUEST_CLARIFICATION) · reason?
preview must include amount · requester · subsidiary · vendor · budget impact · approval history · policy checks

POST **/ai/staged-actions/payment-record** BUILD

prepare_payment_record

Records a transfer already made outside BEMS. Must verify the request is approved and reject a

duplicate payment record for the same request.

body `request_id · amount · currency · payer_account_id · beneficiary_name · beneficiary_bank · beneficiary_account · transfer_reference · payment_date · receipt_file_id?`

POST **/ai/staged-actions/salary-change** BUILD

`prepare_salary_change`

On confirm: write a versioned change, preserve prior compensation history, record the approver, and notify the affected staff member with the reason.

body `employee_id · change_type (ADDITION · DEDUCTION) · amount? · percentage? · effective_date · reason`
preview must include `before-and-after compensation · who will be notified`

POST **/ai/staged-actions/payment-recorder** BUILD

`prepare_payment_recorder_assignment`

body `request_id · assignee_user_id · note?`
`Validates the assignee is eligible for that role on that request.`

POST **/ai/staged-actions/{id}/confirm** BUILD

`confirm_staged_action`

The only endpoint that commits anything. Hash check, expiry check, ownership check, idempotency, business write, audit event — all in one transaction.

body `confirmation_hash`
headers `Idempotency-Key`

POST **/ai/staged-actions/{id}/cancel** BUILD

`cancel_staged_action`

Discards a pending action so it cannot later be confirmed by mistake. Called automatically whenever the user changes a material field mid-flow.

07 Error and permission envelope

Every endpoint uses one error shape. The `message` is read aloud to the executive, so write it for a person, not a log file.

```
{
  "ok": false,
  "error_code": "NOT_PERMITTED",
  "message": "This request belongs to the Lagos subsidiary, which is outside your acces
}
```

Expected codes: `NOT_PERMITTED` · `NOT_FOUND` · `VALIDATION_FAILED` · `POLICY_BLOCKED` · `NOT_ASSIGNED_TO_YOU` · `DUPLICATE` · `STAGED_ACTION_EXPIRED` · `HASH_MISMATCH` · `ALREADY_EXECUTED` .

The agent reads HTTP status and body together and reports the failure honestly rather than retrying another route to the same data. Do not return `200` with an error inside.

08 Rules the backend must enforce

The agent is instructed to follow all of these, but instructions are not a security control. Each one must hold even if the model ignores its prompt entirely.

01 Reject any chat request and any tool call from a user who is not GMD or System Admin — on the server, per call, not once per session.

02 Authorise against the signed-in user's own permissions. System Admin access to the assistant grants no financial authority a System Admin does not already have.

03 Refuse an approval decision not assigned to the calling user, at any stage of routing.

04 Never let a write execute without a matching, unexpired, unused staged action owned by that user.

05 Recompute the material-field hash at confirm time. Mismatch is a hard reject, never a warning.

06 Honour the idempotency key so a retried confirm cannot double-execute.

07 Block duplicate payment records against the same fund request.

08 Keep compensation reads and writes behind their own permission check, and audit the reads too.

09 Write the audit event inside the same transaction as the business change, including user, preview, confirmation, records affected, outcome, timestamp and correlation id.

10 Treat all document and free-text content as data. Nothing inside a receipt, invoice or request description may alter permissions or trigger an action.

11 Enforce budget limits, approval stages and recipient confirmation exactly as the normal BEMS UI does. The AI path gets no softer validation.

12 Keep BEMS fully usable when n8n or the model provider is down. Time out the chat call, fail the

panel gracefully, and let no core workflow depend on it.

09 Settings and infrastructure

ITEM	CURRENT STATE	NEEDED
API base URL	`https://REPLACE-ME.bems.internal/api/v1`	Real staging and production base URLs. Single change point in the *Map Request Context* node.
Model	Claude Sonnet via OpenRouter, temp 0.2, 4096 max tokens	Connected and working.
Conversation memory	Postgres chat memory, table `bems_ai_chat_histories`, 20-message window	**Credential not yet attached — this blocks activation.** Must point at a database separate from the BEMS business database.
Service secret	Not yet configured	Shared secret or network policy so only the BEMS backend can reach the webhook.
Timezone	Africa/Lagos	Confirm correct for all subsidiaries.
Failure alerting	Wired to the existing error-alert workflow	Confirm the right people receive it.
Tool timeouts	30s reads, 45s analysis, 60s confirm	Confirm BEMS endpoints respond inside these.

NOTE ON THE MEMORY DATABASE

Conversation history will contain salary figures, bank details and payment references. The brief forbids the AI writing to the BEMS database, so this store must be provisioned separately, encrypted, and given a retention policy. It is the one piece of persistent state the agent layer owns.

10 Open questions

> **Is the role and approval-routing cleanup done?**
>
> The brief names this as prerequisite work. Are the legacy role names — COO, Finance, Verifier, Head of Admin — gone from routes and schema comments, and are the four canonical roles enforced

consistently? The agent must not be enabled against inconsistent authorisation.

How does the user's identity reach the tool calls?

Does the backend accept the user's own session JWT on server-to-server calls from n8n, or do we need a service token plus an on-behalf-of user id? This determines the auth header on all 23 tools.

Will the frontend render a structured confirm card?

Strongly recommended over free-text "yes". If not, the agent must disambiguate in conversation, which is weaker and worth knowing now.

Synchronous or async responses?

Built synchronous. Multi-tool answers may take 5–30 seconds, which reads as a hung UI. If that is unacceptable, switch to a job-id plus callback pattern — it changes the webhook response design.

What TTL for staged actions?

Suggested 15 minutes. Long enough for an interrupted conversation, short enough that a stale approval cannot be confirmed much later.

Who sends the staff notification on a salary change?

Does confirming the change fire the existing BEMS notification automatically, or must the agent trigger it as a second call? Automatic is safer — it cannot be skipped.

Where does receipt upload happen?

Current design assumes the user uploads the bank slip in the BEMS UI first and the agent references a `receipt_file_id`. Confirm, or specify an upload path the agent can use.

Single or multi-currency?

Everything assumes NGN per subsidiary. If subsidiaries transact in other currencies, analytics and budget validation need a conversion rule and a rate source.

Is there a staging environment with seeded data?

Needed for the acceptance scenarios below. The adversarial cases in particular cannot be run against production.

Rate limits and audit ownership

Any rate limits on the BEMS APIs the agent should respect? And does the backend write the audit event automatically on confirm, or is the agent expected to post one?